

Akindo Backend

Documentación de Módulos

apps/api — FastAPI + SQLAlchemy + Supabase

Módulo: core

Configuración central, base de datos y dependencias globales

¿Qué hace este módulo?

El módulo core es la base de toda la aplicación. Contiene la configuración de variables de entorno, la conexión a la base de datos y las dependencias inyectables globales. Ningún módulo de negocio debe importar directamente de librerías externas sin pasar por core.

Archivos y responsabilidades

config.py	Lee variables de entorno con Pydantic Settings
database.py	Crea el engine de SQLAlchemy y la sesión de BD
dependencies.py	Provee get_db() como dependencia inyectable en routers

Cómo implementarlo

config.py

```
from pydantic_settings import BaseSettings
```

```
class Settings(BaseSettings):
    DATABASE_URL: str
    SUPABASE_URL: str
    SUPABASE_KEY: str
    ENV: str = 'development'

    class Config:
        env_file = '.env'

settings = Settings()
```

database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from app.core.config import settings

engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)

class Base(DeclarativeBase):
    pass
```

dependencies.py

```
from app.core.database import SessionLocal

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

❑ Nota importante

Nunca importes SessionLocal directamente en los routers. Siempre usa Depends(get_db).

Módulo: usuario

Autenticación, registro y perfil base de todos los actores del sistema

¿Qué hace este módulo?

Representa la entidad base del sistema. Tanto Cliente como Distribuidor heredan de Usuario. Este módulo gestiona registro, login, verificación y actualización de perfil. El campo 'tipo' (enum) determina el rol del usuario en el sistema.

Archivos y responsabilidades

models/usuario.py	Tabla USUARIO con id, email, password_hash, tipo, etc.
schemas/usuario.py	UsuarioCreate, UsuarioRead, UsuarioUpdate
repositories/usuario_repository.py	CRUD contra la tabla USUARIO
services/usuario_service.py	Lógica: hashear password, verificar email, validar tipo
routers/usuarios.py	POST /usuarios, GET /usuarios/{id}, PATCH /usuarios/{id}

Cómo implementarlo

models/usuario.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import Enum as SAEnum
from app.core.database import Base

class Usuario(Base):
    __tablename__ = 'usuario'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    nombre: Mapped[str]
    email: Mapped[str] = mapped_column(unique=True)
    password_hash: Mapped[str]
    tipo: Mapped[str] # 'cliente' | 'distribuidor'
    es_verificado: Mapped[bool] = mapped_column(default=False)
```

services/usuario_service.py

```
import bcrypt
from app.schemas.usuario import UsuarioCreate
from app.repositories.usuario_repository import UsuarioRepository

class UsuarioService:
    def __init__(self, repo: UsuarioRepository):
        self.repo = repo

    async def registrar(self, data: UsuarioCreate):
        # Lógica de negocio: hashear password antes de guardar
```

```
hashed = bcrypt.hashpw(data.password.encode(), bcrypt.gensalt())
return await self.repo.crear(data, password hash=hashed.decode())
```

❑ Nota importante

El campo password NUNCA debe aparecer en ningún schema de respuesta (UsuarioRead). Usa model_config con exclude.

Módulo: cliente

Perfil extendido del comprador y gestión de direcciones de entrega

¿Qué hace este módulo?

Cliente es una extensión de Usuario. Tiene una relación 1:1 con USUARIO y puede tener múltiples direcciones de entrega. Este módulo gestiona el perfil del comprador y sus direcciones.

Archivos y responsabilidades

models/cliente.py	Tabla CLIENTE (FK a USUARIO) + DIRECCION_CLIENTE
schemas/cliente.py	ClienteRead, DireccionCreate, DireccionRead
repositories/cliente_repository.py	Queries para cliente y sus direcciones
services/cliente_service.py	Lógica: establecer dirección predeterminada, validar límite
routers/clientes.py	GET /clientes/{id}, POST /clientes/{id}/direcciones

Cómo implementarlo

models/cliente.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import ForeignKey
from app.core.database import Base
```

```

class Cliente(Base):
    __tablename__ = 'cliente'
    usuario_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('usuario.id'),
primary_key=True)
    direcciones: Mapped[list['DireccionCliente']] =
relationship(back_populates='cliente')

class DireccionCliente(Base):
    __tablename__ = 'direccion_cliente'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    cliente_id: Mapped[uuid.UUID] =
mapped_column(ForeignKey('cliente.usuario_id'))
    calle: Mapped[str]
    ciudad: Mapped[str]
    es_predeterminada: Mapped[bool] = mapped_column(default=False)

```

services/cliente_service.py

```

class ClienteService:
    async def establecer_direccion_predeterminada(self, cliente_id, direccion_id):
        # Lógica: desactivar todas las otras direcciones del cliente
        await self.repo.desactivar_direcciones(cliente_id)
        await self.repo.activar_direccion(direccion_id)

```

□ Nota importante

Solo puede haber una dirección con es_predeterminada=True por cliente. Esta lógica vive en el service, no en el router.

Módulo: distribuidor

Perfil del vendedor mayorista y gestión de su catálogo

¿Qué hace este módulo?

Distribuidor es la extensión de Usuario para los vendedores. Tiene RFC, nombre de negocio, valoración promedio y pertenece a categorías. Su catálogo son los productos que ofrece.

Archivos y responsabilidades

models/distribuidor.py	Tabla DISTRIBUIDOR + DISTRIBUIDOR_CON_CATEGORIA
schemas/distribuidor.py	DistribuidorRead, DistribuidorUpdate
repositories/distribuidor_repository.py	Queries con joins a categorías
services/distribuidor_service.py	Lógica: recalcular valoración promedio, asignar categorías
routers/distribuidores.py	GET /distribuidores, GET /distribuidores/{id}, PATCH /distribuidores/{id}

Cómo implementarlo

models/distribuidor.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import ForeignKey, Numeric
from app.core.database import Base

class Distribuidor(Base):
    __tablename__ = 'distribuidor'
    usuario_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('usuario.id'),
primary_key=True)
    rfc: Mapped[str]
    nombre_negocio: Mapped[str]
    valoracion_promedio: Mapped[float] = mapped_column(Numeric(3,2), default=0.0)
    total_valoraciones: Mapped[int] = mapped_column(default=0)
```

services/distribuidor_service.py

```
class DistribuidorService:
    async def recalcular_valoracion(self, distribuidor_id: str):
        # Se llama desde valoracion_service después de una nueva reseña
        promedio, total = await
self.repo.calcular_stats_valoracion(distribuidor_id)
        await self.repo.actualizar_valoracion(distribuidor_id, promedio, total)
```

□ Nota importante

valoracion_promedio y total_valoraciones son campos calculados. Nunca los edites directamente desde el router — siempre pasan por recalcular_valoracion() en el service.

Módulo: producto

Catálogo de productos ofertados por los distribuidores

¿Qué hace este módulo?

Los productos pertenecen a un distribuidor y pueden tener múltiples categorías. Tienen control de existencias, precio (costo), unidad de medida y atributos extra en formato JSON para flexibilidad.

Archivos y responsabilidades

models/producto.py	Tabla PRODUCTO + PRODUCTO_CON_CATEGORIA
schemas/producto.py	ProductoCreate, ProductoRead, ProductoUpdate
repositories/producto_repository.py	Queries con filtros por distribuidor, categoría, disponibilidad
services/producto_service.py	Lógica: validar existencias, toggle disponibilidad, manejar atributos_extra
routers/productos.py	GET /productos, POST /productos, PATCH /productos/{id}, DELETE /productos/{id}

Cómo implementarlo

models/producto.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import ForeignKey, Numeric, JSON
from app.core.database import Base

class Producto(Base):
    __tablename__ = 'producto'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    distribuidor_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('distribuidor.usuario_id'))
    nombre: Mapped[str]
    costo: Mapped[float] = mapped_column(Numeric(10,2))
    medida: Mapped[str]
    existencias: Mapped[int]
    disponible: Mapped[bool] = mapped_column(default=True)
    atributos_extra: Mapped[dict] = mapped_column(JSON, default={})
```

services/producto_service.py

```
class ProductoService:
    async def actualizar_existencias(self, producto_id, cantidad_vendida: int):
        producto = await self.repo.get_by_id(producto_id)
        if producto.existencias < cantidad_vendida:
            raise ValueError('Stock insuficiente')
        nuevo_stock = producto.existencias - cantidad_vendida
        # Si llega a 0, marcar como no disponible automáticamente
        disponible = nuevo_stock > 0
        await self.repo.actualizar(producto_id, existencias=nuevo_stock,
                                   disponible=disponible)
```

❑ Nota importante

El campo `atributos_extra` (JSON) permite guardar propiedades variables por tipo de producto sin alterar el schema. Ejemplo: `{"peso": "1kg", "refrigerado": true}`.

Módulo: categoria

Taxonomía de productos y distribuidores

¿Qué hace este módulo?

Existen dos tipos de categorías en el sistema: `CATEGORIA_PRODUCTO` (para clasificar productos) y `CATEGORIA_DISTRIBUIDOR` (para clasificar el tipo de negocio del distribuidor). Ambas son tablas independientes con sus respectivas tablas de relación muchos-a-muchos.

Archivos y responsabilidades

models/categoria.py	CATEGORIA_PRODUCTO, CATEGORIA_DISTRIBUIDOR y sus tablas pivot
schemas/categoria.py	CategoriaRead, CategoriaCreate
repositories/categoria_repository.py	CRUD y asignación de categorías
services/categoria_service.py	Lógica: prevenir duplicados, validar existencia al asignar
routers/categorias.py	GET /categorias/productos, GET /categorias/distribuidores

Cómo implementarlo

models/categoria.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import ForeignKey
from app.core.database import Base

class CategoriaProducto(Base):
    __tablename__ = 'categoria_producto'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    nombre: Mapped[str] = mapped_column(unique=True)

class ProductoConCategoria(Base):
    __tablename__ = 'producto_con_categoria'
    producto_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('producto.id'),
primary_key=True)
    categoria_id: Mapped[uuid.UUID] =
mapped_column(ForeignKey('categoria_producto.id'), primary_key=True)
```

services/categoria_service.py

```
class CategoriaService:
    async def asignar_a_producto(self, producto_id: str, categoria_ids:
list[str]):
        # Validar que todas las categorías existen antes de asignar
        for cid in categoria_ids:
            existe = await self.repo.existe_categoria_producto(cid)
            if not existe:
                raise ValueError(f'Categoría {cid} no existe')
        await self.repo.asignar_categorias_producto(producto_id, categoria_ids)
```

❑ Nota importante

Las categorías son datos de catálogo — cámbialas poco. Considera seedearlas en la migración inicial de Alembic en vez de crearlas por API.

Módulo: carrito

Carrito de compras temporal por cliente y distribuidor

¿Qué hace este módulo?

El carrito pertenece a un cliente y está segmentado por distribuidor (no se mezclan productos de diferentes distribuidores en el mismo carrito). CARRITO_ITEM almacena los productos y cantidades. El carrito es temporal — se convierte en ORDEN_PEDIDO al hacer checkout.

Archivos y responsabilidades

models/carrito.py	Tabla CARRITO + CARRITO_ITEM
schemas/carrito.py	CarritoRead, CarritoItemAdd, CarritoItemUpdate
repositories/carrito_repository.py	Queries para items, totales, limpiar carrito
services/carrito_service.py	Lógica: validar stock al agregar, calcular totales, regla 1-distribuidor
routers/carrito.py	GET /carrito/{cliente_id}, POST /carrito/{cliente_id}/items, DELETE /carrito/{cliente_id}/items/{item_id}

Cómo implementarlo

models/carrito.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import ForeignKey
from app.core.database import Base

class Carrito(Base):
    __tablename__ = 'carrito'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    cliente_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('cliente.usuario_id'))
    distribuidor_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('distribuidor.usuario_id'))
    items: Mapped[list['CarritoItem']] = relationship(back_populates='carrito')

class CarritoItem(Base):
    __tablename__ = 'carrito_item'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    carrito_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('carrito.id'))
    producto_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('producto.id'))
    cantidad: Mapped[int]
```

services/carrito_service.py

```
class CarritoService:
    async def agregar_item(self, cliente_id, producto_id, cantidad):
```

```

    producto = await self.producto_repo.get_by_id(producto_id)

    # Regla: no se puede agregar más de lo disponible en stock
    if cantidad > producto.existencias:
        raise ValueError('Cantidad supera el stock disponible')

    # Regla: el carrito es por distribuidor, no mezclar
    carrito = await self.repo.get_o_crear(cliente_id,
    producto.distribuidor_id)
    await self.repo.agregar_item(carrito.id, producto.id, cantidad)

```

❑ Nota importante

El carrito NO es una orden. Al hacer checkout, carrito_service llama a orden_service.crear_desde_carrito() y luego limpia el carrito. Nunca elimines el carrito directamente desde el router.

Módulo: orden

Proceso de compra: del checkout a la confirmación del distribuidor

¿Qué hace este módulo?

ORDEN_PEDIDO es el resultado de hacer checkout del carrito. Contiene el estado del proceso de compra (pendiente, confirmada, rechazada). PAQUETE_PEDIDO es el snapshot inmutable de los productos al momento de la compra (precio y medida capturados para que no cambien si el distribuidor edita el producto).

Archivos y responsabilidades

models/orden.py	Tabla ORDEN_PEDIDO + PAQUETE_PEDIDO
schemas/orden.py	OrdenCreate, OrdenRead, OrdenEstadoUpdate
repositories/orden_repository.py	Queries por cliente, distribuidor, estado
services/orden_service.py	Lógica: crear desde carrito, validar stock final, cambios de estado
routers/ordenes.py	POST /ordenes, GET /ordenes/{id}, PATCH /ordenes/{id}/estado

Cómo implementarlo

models/orden.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import ForeignKey, Numeric, Enum as SAEnum
from app.core.database import Base
import enum

class EstadoOrden(str, enum.Enum):
    PENDIENTE = 'pendiente'
    CONFIRMADA = 'confirmada'
    RECHAZADA = 'rechazada'

class OrdenPedido(Base):
    __tablename__ = 'orden_pedido'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    cliente_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('cliente.usuario_id'))
    distribuidor_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('distribuidor.usuario_id'))
    direccion_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('direccion_cliente.id'))
    estado: Mapped[EstadoOrden] = mapped_column(default=EstadoOrden.PENDIENTE)
    motivo_rechazo: Mapped[str | None]

class PaquetePedido(Base):
    __tablename__ = 'paquete_pedido'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    orden_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('orden_pedido.id'))
    producto_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('producto.id'))
    cantidad: Mapped[int]
    costo_unitario: Mapped[float] = mapped_column(Numeric(10,2)) # snapshot
    medida_snapshot: Mapped[str] # snapshot
```

services/orden_service.py

```
class OrdenService:
    async def crear_desde_carrito(self, cliente_id, carrito_id, direccion_id):
        items = await self.carrito_repo.get_items(carrito_id)

        # Snapshot: capturar precios actuales antes de crear la orden
        paquetes = []
        for item in items:
            producto = await self.producto_repo.get_by_id(item.producto_id)
            paquetes.append({
                'producto id': producto.id,
```

```

        'cantidad': item.cantidad,
        'costo_unitario': producto.costo, # precio en este momento
        'medida_snapshot': producto.medida
    })

    orden = await self.repo.crear(cliente_id, direccion_id, paquetes)
    await self.carrito_repo.limpiar(carrito_id) # vaciar carrito
    return orden

```

❑ Nota importante

PAQUETE_PEDIDO es inmutable por diseño. Guarda el precio y medida al momento de compra. Nunca actualices costo_unitario ni medida_snapshot después de creados.

Módulo: pedido

Seguimiento post-confirmación: entrega y comisiones

¿Qué hace este módulo?

PEDIDO se crea cuando una ORDEN_PEDIDO es confirmada por el distribuidor. Representa el estado de entrega del pedido, el total final y la comisión del servicio. Es el módulo que eventualmente se integrará con el sistema externo de Entregas.

Archivos y responsabilidades

models/pedido.py	Tabla PEDIDO con estado, total y comisión
schemas/pedido.py	PedidoRead, PedidoEstadoUpdate
repositories/pedido_repository.py	Queries por estado, por cliente, por distribuidor
services/pedido_service.py	Lógica: calcular comisión, transiciones de estado, trigger a Entregas
routers/pedidos.py	GET /pedidos/{id}, PATCH /pedidos/{id}/estado

Cómo implementarlo

models/pedido.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import ForeignKey, Numeric, Enum as SAEnum
from app.core.database import Base
from datetime import datetime
import enum

class EstadoPedido(str, enum.Enum):
    EN_PREPARACION = 'en_preparacion'
    EN_CAMINO = 'en_camino'
    ENTREGADO = 'entregado'

class Pedido(Base):
    __tablename__ = 'pedido'
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    orden_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('orden_pedido.id'),
unique=True)
    estado: Mapped[EstadoPedido]
    total: Mapped[float] = mapped_column(Numeric(10,2))
    comision_servicio: Mapped[float] = mapped_column(Numeric(10,2))
    confirmado_at: Mapped[datetime | None]
    entregado_at: Mapped[datetime | None]
```

services/pedido_service.py

```
TASA_COMISION = 0.05 # 5%

class PedidoService:
    async def crear_desde_orden(self, orden_id):
        paquetes = await self.orden_repo.get_paquetes(orden_id)
        total = sum(p.costeo_unitario * p.cantidad for p in paquetes)
        comision = round(total * TASA_COMISION, 2)

        return await self.repo.crear(
            orden_id=orden_id,
            total=total,
            comision_servicio=comision
        )

    # PUNTO DE EXTENSIÓN: en el futuro, llamar al sistema de Entregas aquí
    async def notificar_entrega(self, pedido_id):
        # TODO: integrar con sistema externo de Entregas
        pass
```

□ Nota importante

La constante TASA_COMISION debe moverse a config.py cuando esté lista la configuración. El método notificar_entrega() es el punto de extensión hacia el sistema externo de Entregas del diagrama C4.

Módulo: valoracion

Sistema de reseñas de clientes a distribuidores

¿Qué hace este módulo?

Un cliente puede valorar a un distribuidor una vez por pedido entregado. La valoración tiene puntuación (1-5) y comentario opcional. Después de cada valoración, se recalcula el valoracion_promedio del distribuidor.

Archivos y responsabilidades

models/valoracion.py	Tabla VALORACION con FK a pedido, cliente y distribuidor
schemas/valoracion.py	ValoracionCreate, ValoracionRead
repositories/valoracion_repository.py	CRUD y verificación de valoración existente por pedido
services/valoracion_service.py	Lógica: validar que pedido fue entregado, prevenir duplicados, trigger recálculo
routers/valoraciones.py	POST /valoraciones, GET /distribuidores/{id}/valoraciones

Cómo implementarlo

models/valoracion.py

```
import uuid
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import ForeignKey, CheckConstraint
from app.core.database import Base
from datetime import datetime

class Valoracion(Base):
    __tablename__ = 'valoracion'
    table_args = (
```

```

        CheckConstraint('puntuacion >= 1 AND puntuacion <= 5',
name='check_puntuacion'),
    )
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    pedido_id: Mapped[uuid.UUID] = mapped_column(ForeignKey('pedido.id'),
unique=True)
    cliente_id: Mapped[uuid.UUID] =
mapped_column(ForeignKey('cliente.usuario_id'))
    distribuidor_id: Mapped[uuid.UUID] =
mapped_column(ForeignKey('distribuidor.usuario_id'))
    puntuacion: Mapped[int]
    comentario: Mapped[str | None]
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)

```

services/valoracion_service.py

```

class ValoracionService:
    async def crear(self, data: ValoracionCreate):
        # Regla 1: solo se puede valorar un pedido entregado
        pedido = await self.pedido_repo.get_by_id(data.pedido_id)
        if pedido.estado != 'entregado':
            raise ValueError('Solo puedes valorar pedidos entregados')

        # Regla 2: un pedido solo puede tener una valoración
        ya_existe = await self.repo.existe_por_pedido(data.pedido_id)
        if ya_existe:
            raise ValueError('Este pedido ya fue valorado')

        valoracion = await self.repo.crear(data)

        # Trigger: recalcular promedio del distribuidor
        await
self.distribuidor_service.recalcular_valoracion(data.distribuidor_id)
        return valoracion

```

□ Nota importante

La constraint unique en pedido_id a nivel de base de datos es el respaldo de seguridad, pero la validación de duplicado debe ocurrir en el service primero para dar un mensaje de error claro al usuario.